

GROUP NUMBER: 89

GROUP MEMBERS: Crucioli Giuliano, Dazzini Bartolo, Gotti Paolo

AVAILABLE INPUTS: Input files are available in the hdfs file system: /data/BDC2425/artificial1M7D100K.txt and /data/BDC2425/artificial4M7D100K.txt

PART 1: The goal of this test is to assess the scalability of the standard and fair implementations. The test must be performed on file artificial4M7D100K.txt. However, if your implementation is slow (i.e., taking more than 10 minutes for the slowest run), you can use the smaller file artificial1M7D100K.txt. You must use the following parameters: $L=16$, $K=100$, $M=10$.

Fill in the following table.

Name of used file: /data/BDC2425/artificial4M7D100K.txt

SCALABILITY WITH RESPECT TO NUMBER OF EXECUTORS			
Number of executors	Spark Lloyd's implementation	MRFairLloyd	MRComputeFairObjective
2	17492 ms	87690 ms	9287 ms
4	15343 ms	46109 ms	4806 ms
8	11191 ms	23463 ms	2562 ms
16	5748 ms	14127 ms	1689 ms

General hints:

- Remember that Spark uses the lazy evaluation for constructing an RDD. Therefore, be sure to include an action on the final RDD when you take running times.
- Any used RDD in your program should be cached.
- Do not include the reading of the input in your running times.

PART 2: Describe the program GxxGEN.java or GxxGEN.py that you have implemented for point 5 of the specifications. Include a brief high-level description of your program and the constraints, if any, on the input parameters (e.g., the minimum number of points N).

To implement the program G89GEN.java we took the cue from the paper regarding Socially Fair k-means Clustering.

If we consider just one cluster, the idea is to put just one point of class "A" in position $(-10, 0)$ and all the remaining $N - 1$ points in position $(10, 0)$ with label "B". For this reason, the **minimum number of points** that is needed to see a clear difference in the loss functions for K clusters is $N = 3K$ (so in each cluster you have one point of class A and two points of class B).

By placing the points in the way described above, the Lloyd's algorithm won't consider the demographic classes and so it will produce a centroid very unbalanced towards class "B", instead the Fair algorithm will give more importance to the point of class "A" by producing a centroid a little closer to the latter.

If we want more clusters, we reproduce the exact setting, but we place the points with a vertical space of 30 above the previous generated ones. So, the points of the i -th cluster will be in position $(-10, 30 \cdot (i-1))$ (class "A") and $(10, 30 \cdot (i-1))$ (class "B").

By doing so the Lloyd's algorithm produces centroids that discriminate the minority class "A", thus producing a higher Fair Objective with respect to the Fair variant.

The specific positions of the points is not important, but the key is the proportion among the horizontal distances between points of different class and the vertical distance between points belonging to the same class. In particular,

to show a clear difference in the loss functions, the vertical distance should be greater than the horizontal one. We've noticed that, if the horizontal distance between points is 20 and the vertical distance is 21, then if we use $N = 3K$, the fair loss computed with the centroids produced by the Lloyd's algorithm is about 170, meanwhile the one computed with fair centroids is about 100.

If we increase the vertical distance and we set it to 30, we notice that the variance of the values of the two losses around their mean values becomes almost zero: the loss computed on the points produced by the standard Lloyd's stabilizes on 177 and the one related to the fair algorithm is 100

If we increase more the vertical distance the two losses remain the same. In order to have a greater difference between the losses, we need to use a larger N .